FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Introducción a la Programación

Diagramas de Flujo

Docentes:
Ing. Sebastián Balestrini
Esp. Ing. Gustavo Di Stefano

Introducción a la Programación

¿Qué es programar?

Programar es el proceso de crear un conjunto de instrucciones, llamado programa, que le indica a una computadora cómo realizar una tarea específica. Estas instrucciones deben estar escritas en un lenguaje que la máquina pueda interpretar y ejecutar.

¿Qué es un algoritmo?

Un algoritmo es una secuencia finita, precisa y ordenada de pasos que permiten resolver un problema o realizar una tarea.

Características fundamentales:

- **Finito**: Debe tener un número limitado de pasos.
- **Preciso**: Cada instrucción debe estar claramente definida.
- **Ordenado**: Las acciones deben ejecutarse en un orden lógico.

Lenguajes de Programación

Los lenguajes de programación son herramientas formales utilizadas para escribir los algoritmos que serán interpretados por una computadora. Se clasifican jerárquicamente según su nivel de abstracción:

1. **Lenguaje máquina**
    - Está compuesto por instrucciones binarias (0 y 1).
    - Es específico de cada procesador.
    - Ejemplo: 10110000 01100001

2. **Lenguaje de bajo nivel (ensamblador)**
    - Usa mnemónicos (como MOV, ADD) y es dependiente del hardware.
    - Requiere conocimiento detallado de la arquitectura.
    - Ejemplo: MOV AL, 61h
3. **Lenguaje de alto nivel**
    - Su sintaxis es más cercana al lenguaje humano.
    - Es independiente del hardware.
    - Ejemplos: C, Python, Java, Pascal.

Lenguajes Compilados vs. Interpretados

Antes de ejecutarse, un programa debe ser traducido a lenguaje máquina. Esto se puede hacer de dos formas:

Compilados: Se traducen completamente antes de ejecutarse.

- Ejemplo: C, C++
- Ventaja: Mayor rendimiento al ejecutarse.
- Desventaja: Menor flexibilidad para pruebas rápidas.

Interpretados: Se traducen línea por línea durante la ejecución.

- Ejemplo: Python, JavaScript
- Ventaja: Permite pruebas y modificaciones rápidas.
- Desventaja: Menor eficiencia en ejecución.

Estilos de Programación

Los estilos de programación definen cómo se estructura y organiza el código. En esta unidad se presentan:

- **Lineal:** Instrucciones ejecutadas en secuencia.
- **Estructurada:** Usa estructuras básicas como secuencia, decisión y repetición.
- **Modular:** Divide el código en módulos o funciones independientes.
- **Orientada a objetos:** Organiza el código en objetos que representan entidades con atributos y comportamientos.

Proceso de Desarrollo de Software

Este proceso abarca desde la concepción de una idea hasta su implementación y mantenimiento. Las fases son:

- **Definición del problema**: Entender qué debe resolver el software.
- **Análisis**: Identificar entradas, salidas esperadas y procesamiento necesario.
- **Diseño del algoritmo**: Planificar la solución con herramientas como diagramas de flujo o pseudocódigo.
- **Codificación**: Escribir el código fuente.
- **Prueba y depuración**: Verificar que funcione correctamente y corregir errores.
- **Documentación**: Describir el funcionamiento y propósito del programa.
- **Mantenimiento**: Actualizar, corregir y mejorar el software después de su entrega.

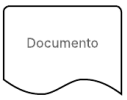
¿Qué es un Diagrama de Flujo?

Un diagrama de flujo es una representación gráfica de un algoritmo o proceso. Usa símbolos estándar conectados por flechas que indican el flujo de ejecución.

¿Para qué se usa?

- Para planificar algoritmos antes de programarlos.
- Para visualizar la lógica de resolución de un problema.
- Para comunicar soluciones a otros.

Principales Símbolos

Símbolo	Nombre	Función
	Flecha	Indica el flujo del algoritmo.
	Inicio / Fin	Representa el inicio o el fin del diagrama.
	Proceso	Representa una instrucción o acción. Ej: "x = x + 1"
	Decisión	Representa una condición que puede ser verdadera o falsa. Ej: "¿x > 10?"
	Entrada/Salida	Representa leer datos o mostrar datos
	Documento	Un documento impreso
	Base de datos	Representa una base de datos

Símbolo	Nombre	Función
	Comentario	Comentario o Anotación

Buenas Prácticas

1. Un solo inicio y fin.
2. Las decisiones deben tener dos salidas: Sí y No.
3. Usar nombres descriptivos para variables y acciones.
4. El flujo debe ser claro de arriba hacia abajo y de izquierda a derecha.
5. Evitar cruces de flechas cuando sea posible.
6. Declarar variables antes de usarlas

Herramientas:

- **Flowrun.io**: https://flowrun.io/

Es una herramienta online, gratuita y sin necesidad de instalación, diseñada específicamente para la creación de diagramas de flujo de manera intuitiva, simple y visual.

Una de sus principales ventajas es que no solo permite construir el diagrama paso a paso, sino que además ofrece funcionalidades avanzadas como:

- Ejecución directa del diagrama, mostrando el recorrido lógico paso a paso y los resultados en una consola integrada, lo cual permite verificar el funcionamiento del algoritmo sin necesidad de codificarlo aún.
- Visualización automática del código fuente generado, traducido a múltiples lenguajes de programación como C, C++, Java, Python, JavaScript, PHP, entre otros.

- **Lucidchart**: https://www.lucidchart.com/
- **AppDiagrams**: https://app.diagrams.net/

Es otra herramienta online que permite diseñar diversos tipos de diagramas, incluyendo diagramas de flujo, diagramas UML, mapas conceptuales, organigramas y más.

A diferencia de Flowrun.io, estas herramientas se enfocan en la representación visual y no en la ejecución del algoritmo, por lo tanto no muestran resultados en consola ni generan código automáticamente, pero es muy útil para documentación y presentación profesional de diagramas.

Nota aclaratoria sobre los símbolos en Flowrun.io

En la herramienta Flowrun.io, los símbolos utilizados para representar la entrada y salida de datos no siguen exactamente la convención estándar de los diagramas de flujo.

En los diagramas tradicionales se utiliza un paralelogramo para representar entrada/salida (como "ingresar datos" o "mostrar resultados").



En **Flowrun.io** se utiliza:

- La entrada se representa con un trapecio invertido.
- La salida se representa con un trapecio normal.

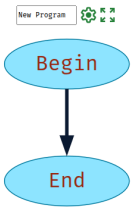


Importante: *Aunque el símbolo es diferente, su función es la misma*. Esto se hace simplemente por un tema de diseño visual en la herramienta, pero no cambia la lógica del diagrama.

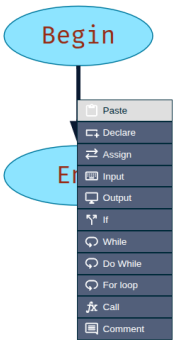
Cómo usar Flowrun.io – Guía Paso a Paso

1. Ingresar al sitio: Abrir el navegador en flowrun.io. No requiere registro. Podemos loguearnos con una cuenta de GitHub o Google.

2. Agregar bloques



- Al hacer click sobre la flecha nos ofrece un menú contextual para agregar un nuevo bloque.



- Seleccioná el tipo: Inicio, Proceso, Condición, Entrada/Salida, Fin.

3. Editar bloques

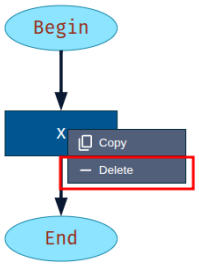
- Hacé doble clic en el texto del bloque para modificarlo.
- Cambiá el color o la forma desde el menú lateral.

4. Conectar bloques

- Hacé clic en una flecha desde un bloque y arrastrala hasta otro bloque.
- Podés etiquetar las ramas de una condición con “Sí” y “No”.

5. Borrar bloques

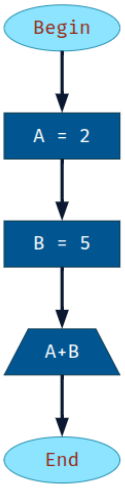
- Hacé click derecho sobre el bloque



6. Guardar o compartir

- Usá el botón Export para descargar la imagen.
- También podés compartir el enlace del diagrama.

Ejemplo 1: Sumar dos números

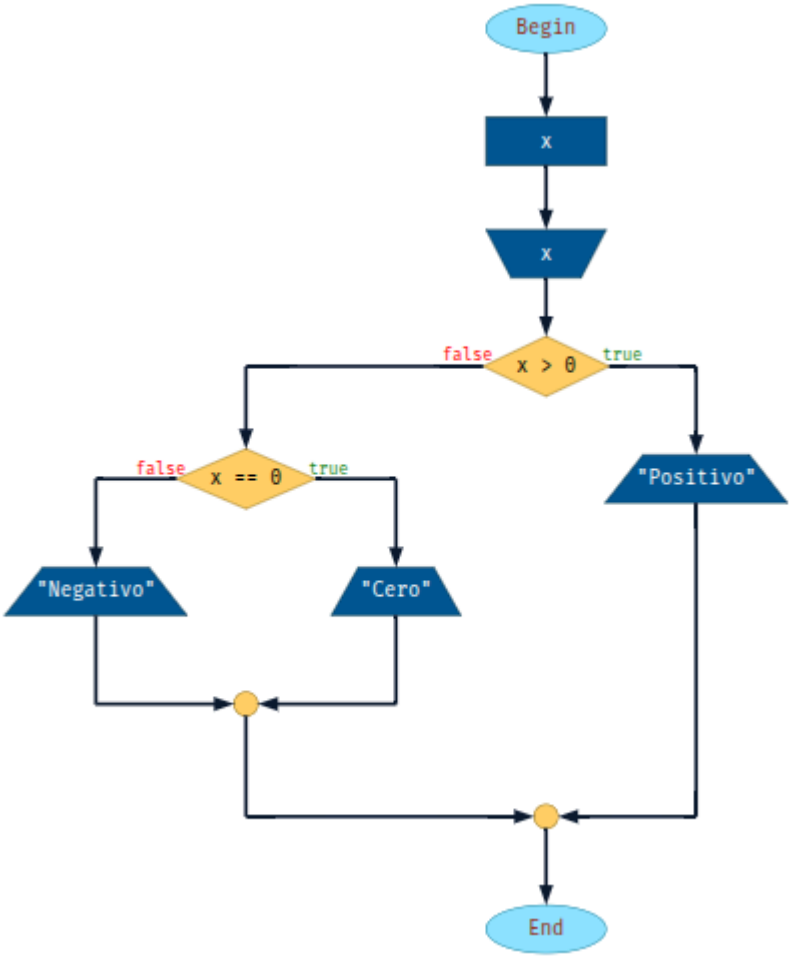



```
[Started at: 2025-07-11 01:11:34]
7
[Finished at: 2025-07-11 01:11:34]
[Execution time: 0.095S]
```

Este diagrama de flujo representa un algoritmo simple para sumar dos números enteros y mostrar el resultado. Se compone de los siguientes pasos:

1. Inicio (Begin): Indica el comienzo del algoritmo.
2. Asignación de A = 2: Se inicializa la variable A con el valor 2.
3. Asignación de B = 5: Se inicializa la variable B con el valor 5.
4. Cálculo de A + B: Se realiza la operación de suma. En este caso, se suman los valores 2 y 5.
5. Fin (End): Marca el final del algoritmo.

Ejemplo 2: Dado un número ingresado, verificar si es Positivo, Negativo o Cero.



Este diagrama de flujo permite clasificar un número x en función de su valor:

1. Inicio (Begin): Marca el inicio del algoritmo.

2. Declaración de la variable a usar (x)

3. Lectura o asignación de x : El algoritmo recibe un número ingresado por el usuario.

4. Primera decisión: x > 0: Se evalúa si x es mayor a cero:

- Si es verdadero, se muestra el mensaje "Positivo" y se continúa al final.
- Si es falso, se pasa a una segunda condición.

5. Segunda decisión: x == 0; Si x no fue mayor que cero, se evalúa si es igual a cero:

- Si es verdadero, se muestra "Cero".
- Si es falso, entonces es menor que cero, y se muestra "Negativo".

6. Fin (End): Todos los caminos del flujo convergen aquí, marcando el final del proceso.